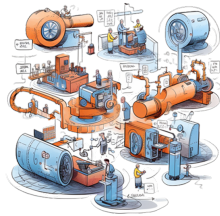


The Information Propagation Framework

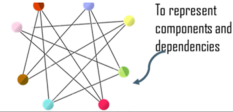
A multi-metric toolkit for efficient performance profiling of DAG networks.

The Information Propagation Framework

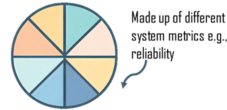


Physical System

**Information
Propagation
Algorithms**



System Model



System Performance Profile

A **Julia-based** framework for network reliability analysis and information propagation that offers a collection of algorithms for analysing how information such as failures, time, cost, capacity or other factors propagate through complex networks that can be modelled as Directed Acyclic Graphs (DAGs).

The Core Idea

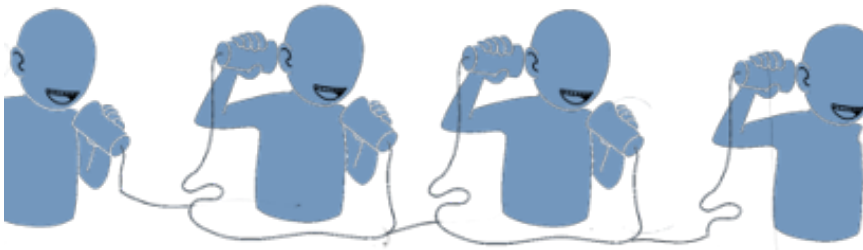
The Framework is based on two fundamental concepts:

① **Topological Sorting of DAGs**

- Provides the fixed processing order of nodes in the network as Iteration sets
- Guarantees dependencies are resolved before dependent nodes
- Natural parallelisation boundaries

② **Iterative Message Passing**

- Information starts from sources to sinks in ordered layers
- Information is computed directly from all dependencies



Why DAGs? The Mathematical Advantage

Directed Acyclic Graphs provide:

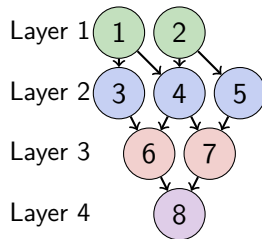
- **Computational Guarantee:** No cycles = No infinite loops
- **Optimal Substructure:** Local solutions build to global optimum
- **Natural Parallelization:** Nodes in same topological layer are independent
- **Exact Solutions:** No need for iterative convergence

Mathematical Complexity For Most Algos:

$$\text{Time Complexity} = O(V + E) \quad (1)$$

$$\text{Space Complexity} = O(V + E) \quad (2)$$

$$\text{Parallelization} = O(\log V) \quad (3)$$



Topological layers enable parallel processing within each level

What If you Don't have DAG Models?

Setting the Stage for Network Analysis

GenerateGraphModule.jl

- Infrastructure-style DAG generation
- Realistic network topologies
- Configurable parameters
- Multiple output formats

UndirectedToDagModule.jl

- Undirected → DAG conversion
- Multi-metric importance scoring
- Dynamic cycle breaking
- As much edge retention as possible

Why These Matter: We couldn't find enough real DAG structures to test with!

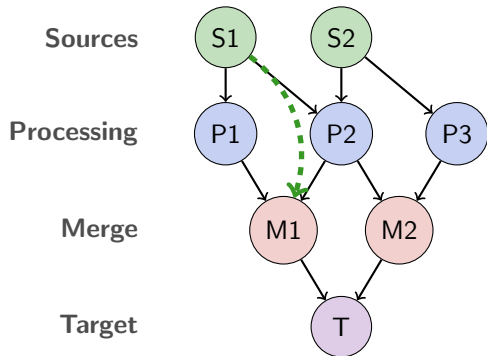
GenerateGraphModule.jl - Creating Realistic Test Networks

Key Features:

- **Ranked Nodes** - Sources, processing layers, sinks etc
- **Critical Path Generation** - Connectivity with skip connections
- **Redundancy Control** - Configurable backup paths
- **Fork/Join Diamond Patterns** - Realistic branching and merging.

Why This Matters:

- Creates realistic test scenarios
- Tries to mimic real infrastructure networks



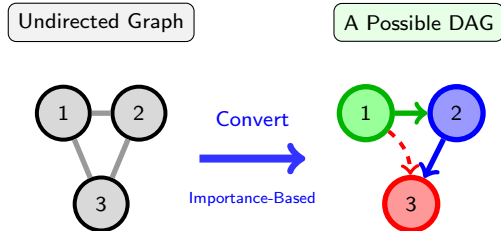
UndirectedToDagModule.jl - Graph Conversion

Multi-Metric Importance Scoring:

- **Degree Centrality** (50%) - High-traffic nodes get priority for edge retention
- **Eigenvector Centrality** (30%) - Considers network influence: nodes connected to high-traffic nodes matter more
- **Clustering Coefficient** (20%) - Aims to preserve Local structure

Cycle Breaking Flow:

- 1 Edge direction by importance ranking
- 2 DFS-based cycle detection
- 3 Remove lowest-importance edges in cycles
- 4 Validate & Try again



FaultTreeModule.jl - From Failure Models to System Analysis

Fault Tree (Failure Logic)

AND Gate (Both must fail)



INVERT
Parallel Success

OR Gate (Either can fail)



INVERT
Series Success

RBD (Success Logic)

Parallel Configuration



MAP
Multiple Paths

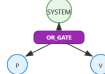
Series Configuration



MAP
Sequential Path

DAG (Reachability)

OR Join Node



Sequential Chain



Value Proposition:

- **FT as System Model** – Use fault trees as input format to capture system structure
- **Multiple Analysis Types** – Same system structure, different analysis
- **Beyond Failure Analysis** – Process dynamics, capacity, critical paths, flow

FaultTreeModule.jl - From Failure Models to System Analysis

Fault Tree (Failure Logic)

AND Gate (Both must fail)



INVERT
Parallel Success

OR Gate (Either can fail)



INVERT
Series Success

RBD (Success Logic)

Parallel Configuration



MAP
Multiple Paths

Series Configuration



MAP
Sequential Path

DAG (Reachability)

OR Join Node



Sequential Chain

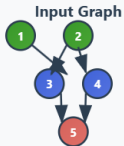


Conversion Process:

- **Parse FT Structure** – Extract system components and relationships
- **Logic Inversion** – Convert failure logic to success logic (FT → RBD → DAG)
- **DAG Generation** – Create unified graph structure for framework algorithms

Input Processing Module

Compute Once, Use Everywhere - Single graph structure reused across all Modules



CSV Format

```
0.8,0.0,5.0,3.0,0
0.9,0.4,0.0,6.0,0
```

Input Processing Module

read_graph_to_dict()

Complete Output Data Structures

edgelist

Vector[(1,3), (1,4), (2,3), (2,4), (3,5), (4,5)]

fork_nodes

Set[Int64]
{1, 2} (nodes with > 1 outgoing)

outgoing_index

Dict[Int64, Set[Int64]]
1=>{3,4}, 2=>{3,4}, 3=>{5}, 4=>{5}

join_nodes

Set[Int64]
{3, 4, 5} (nodes with > 1 incoming)

incoming_index

Dict[Int64, Set[Int64]]
3=>{1,2}, 4=>{1,2}, 5=>{3,4}

iteration_sets

Vector(Set[Int64])
[{1,2}, {3,4}, {5}]

source_nodes

Set[Int64]
{1, 2}

ancestors

Dict[Int64, Set[Int64]]
5=>{1,2,3,4,5}

descendants

Dict[Int64, Set[Int64]]
1=>{3,4,5}

Input Processing Module

Compute Once, Use Everywhere - Single graph structure reused across all Modules

Graph Structure

(edgelist, outgoing_index, incoming_index):

- $O(E)$ space complexity -so fast lookups
- Unified representation for all modules

Pattern Recognition

(fork_nodes, join_nodes):

- Great for diamond pattern analysis
- Supports redundancy analysis

Topological Structure

(iteration_sets, source_nodes):

- Great for parallel processing within levels
- Guarantees dependency resolution order
- Natural parallelisation boundaries

Relationship Tracking

(ancestors, descendants):

- Precomputed for $O(1)$ relationship lookups
- Great for reachability analysis
- More efficient path finding

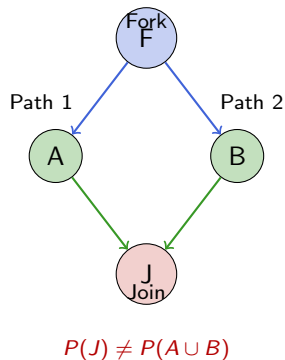
Diamond Definition and Mathematical Impact

Diamond Structure Components:

- **Fork Node (F):** Multi-child branching point
- **Join Node (J):** Multi-parent convergence point
- **Divergent Paths:** ≥ 2 distinct paths $F \rightarrow J$
- **Shared Ancestry:** Common fork ancestor

Why They Matter:

- Break independence assumptions
- Classical "Noisy-OR" fails
- Requires exact conditioning on fork states
- Plus easy parallelisation opportunities



Diamond Classification System - 6 Types

By Structure Complexity:

- ① **Basic Induced:** Simple fork-join (1-3 forks)
- ② **Self-influenced:** Fork connects directly to join
- ③ **Multi-fork:** Multiple fork nodes → same join

By Advanced Patterns:

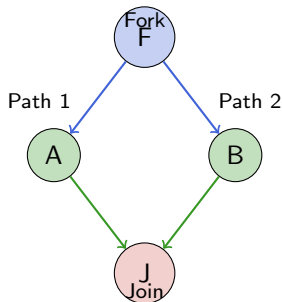
- ④ **Interconnected:** Diamonds sharing nodes/edges
- ⑤ **Nested:** Diamonds within diamonds
- ⑥ **Chained:** Sequential diamond structures

Classification Dimensions:

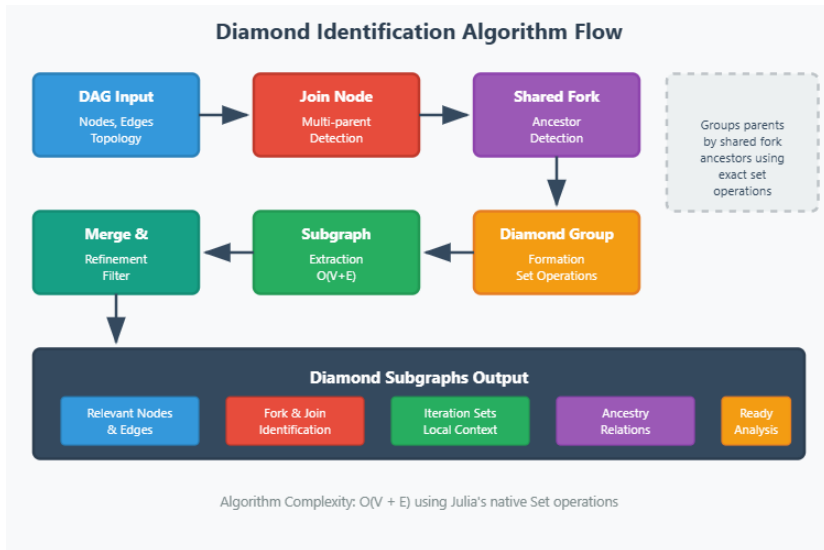
- **Fork Structure:**
Single/Multi/Chained/Self-influence
- **Internal Structure:**
Simple/Nested/Sequential/Interconnected
- **Path Topology:**
Parallel/Converging/Branching/Cross-connected
- **Join Structure:** Single/Hierarchical/Partial
- **External Connectivity:**
Isolated/Bridge/Embedded

2.2 Network Decomposition Algorithm

- 1 **Shared Ancestor Detection:** Fork influence mapping
- 2 **Diamond Group Formation:** Parent grouping by fork ancestry
- 3 **Subgraph Extraction:** Complete diamond context
 - All relevant nodes and edges
 - Conditioning nodes
 - Local iteration sets for processing
 - Complete ancestry relationships



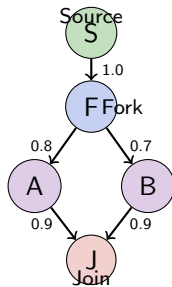
Diamond Identification Flow



Reachability Analysis

The Problem:

- Calculate exact probability of node activation from source nodes
- Handle multiple dependency paths without double-counting
- Classical "Noisy-OR" overestimates shared ancestor influence
- Diamond structures break independence assumptions

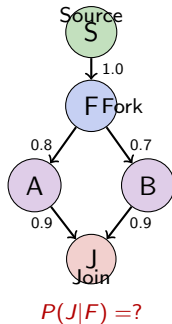


Reachability Analysis - The Diamond Problem

The Challenge: Simple Noisy-OR combination overestimates when shared dependencies

Our Solution:

- **State conditioning** on fork nodes
- **Inclusion-exclusion** for overlapping paths
- **Cutset extraction** minimizes conditioning complexity
- **Multiple algorithms** - same math, different probability representations



Reachability Algorithm Family - 5 Different Approaches

Four Exact Algorithms:

- **Float:** Single probability values
- **Interval:** $[p_{min}, p_{max}]$ bounds
- **P-box:** Full probability distributions

One Approximate:

- **Monte Carlo:** Statistical validation

Selection Criteria:

- **Uncertainty needs:** None, bounds, or full distributions
- **Memory constraints:** 8 bytes to 8KB per probability
- **Speed requirements:** Fast to slow
- **Network size:** Small to unlimited scale
- **Validation needs:** Cross-verification capability

Key Point: Same math, different probability representations. Choice depends on the needs for uncertainty and computational constraints.

3.1 Reachability Algorithm Family - Mathematical Implementation

Mathematical Implementation:

- ① **Regular Paths:** Sum parent contributions

$$P_{\text{parent}}(j) = \sum_{i \in \text{parents}(j)} P(i) \cdot P(i \rightarrow j) \quad (4)$$

- ② **Multiple Paths:** Apply inclusion-exclusion

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \quad (5)$$

- ③ **Diamond Conditioning:** Enumerate fork states

$$P(\text{join}) = \sum_{s \in \{0,1\}^{|F|}} P(\text{join}|s) \cdot P(s) \quad (6)$$

where F = fork nodes, s = state vector

Reachability Algorithm Family - Input Types

Same Probability Ops, Different probability representations

Float Reachability (3.1):

- Standard Float64 values
- Example: $P = 0.75$
- $O(1)$ Direct arithmetic operations

Interval Reachability (3.2):

- Probability bounds: $[p_{min}, p_{max}]$
- Example: $P \in [0.6, 0.8]$
- $O(1)$ interval arithmetic

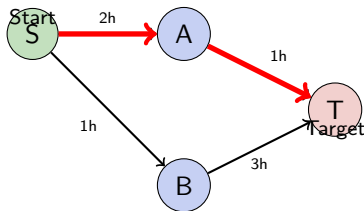
P-box Reachability (3.3):

- Full probability distributions (via PBA.jl)
- Example: $P \sim \text{Beta}(2, 3)$ or $\text{Normal}(\mu, \sigma)$
- Complete uncertainty characterisation
- Convolution-based arithmetic

4. Critical Path Analysis

Challenges:

- **Find longest/critical path** — “What’s the earliest completion time?”
- **Determine completion time and max cost** — “What’s the maximum cost path?”
- **Handle multiple criteria (time, cost, risk)** — “What’s the worst-case accumulated risk?”
- **Identify critical tasks/activities** — “What’s the worst-case risk accumulation?”



Node Values:

- **S:** Time=0, Cost=0
- **A:** Time=5, Cost=100
- **B:** Time=3, Cost=150
- **T:** Time=2, Cost=50

Critical Path Analysis - Mathematical Foundation

Generalized Algorithm Framework:

- ① **Topological Processing:** Use iteration sets for dependency resolution
- ② **Three-Stage Computation:** For each node in topological order:

Stage 1: Propagate(*parent_value*, *edge_value*) (7)

Stage 2: Combine(*propagated_values*) (8)

Stage 3: Apply Node Processing(*combined_input*, *node_value*) (9)

Specific Algorithm Instantiations:

Analysis Type	Combination	Propagation	Node Function
Time Critical Path	$\max()$	$parent + edge$	$input + duration$
Cost Analysis	$\max()$	$parent + edge$	$input + cost$
Risk Analysis	$\max()$	$parent + edge$	$input + risk$

Critical Path Analysis - Input Types and Flexibility

Time Analysis Inputs:

- `task_durations`: Duration per node
- `edge_delays`: Dependency delays
- `start_time`: Project start time
- Multiple time units supported (hours, days, etc.)
- `NonNegativeTime` for exact calculations

Cost Analysis Inputs:

- `node_costs`: Processing costs per node
- `edge_costs`: Transmission/setup costs
- `start_cost`: Initial project cost

Risk Analysis Inputs:

- `node_risk`: Risk contribution per node
- `edge_risk`: Dependency risk contribution
- `start_risk`: Initial risk baseline

Custom Analysis:

- Define custom combination functions
- Define custom propagation functions
- Define custom node functions
- Support any numeric type
- Configurable optimization objectives

All $O(V + E)$ complexity using iteration sets from Input Module

Capacity Analysis – Overview

FlowCapacity = exact network-flow analysis on DAG networks

Core call:

- `analyze_all(edgelist, outgoing_index, incoming_index, capacities, source_nodes, sink_nodes; ...)`
- Baseline max-flow solved **once**
- Downstream modules reuse the same baseline solve

Optional node constraints:

- `node_capacities` enables node-capacitated analysis
- If omitted, node-capacitated stage is skipped

What it returns :

- `flow.max_flow`, sink flows, saturated edges
- Min-cut sets + edges in some/every min-cut
- SPOF edges/nodes, bottleneck ranking, redundancy
- Failure impact (single-edge, k-edge, scenarios)
- Sensitivity (critical edges, marginal capacity, Birnbaum)
- Thresholds (degradation / upgrade)

Capacity – Key Flow Equations

1. Edge-capacitated max flow (baseline):

$$\begin{aligned} \max \quad & \sum_{(s,v) \in E} f_{sv} \\ \text{s.t.} \quad & 0 \leq f_{uv} \leq c_{uv}, \quad \forall (u,v) \in E \\ & \sum_{u:(u,v) \in E} f_{uv} = \sum_{w:(v,w) \in E} f_{vw}, \quad \forall v \notin \{s,t\} \end{aligned}$$

2. Max-Flow Min-Cut theorem (exactness guarantee):

$$\max \text{flow}(s \rightarrow t) = \min \text{cut_capacity}(s, t)$$

3. Node-capacitated flow via node-splitting:

$$\forall v : \quad v_{in} \rightarrow v_{out} \text{ with capacity } c(v), \quad f(v_{in}, v_{out}) \leq c(v)$$

Used by: flow solve, min-cut, SPOF detection, threshold and failure-impact analysis

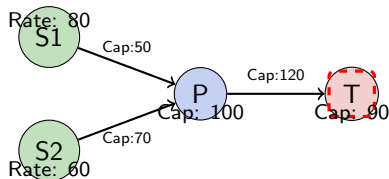
Capacity Analysis – Parameters and Outputs

Required Inputs:

- edgelist
- outgoing_index, incoming_index
- capacities map: $(u,v) \Rightarrow \text{Float64}$
- source_nodes, sink_nodes

Key Options:

- algorithm = :dinic (or Edmonds–Karp / Push–Relabel)
- target_flow, k_failure
- cut_limit, path_limit, tol
- Optional: node_capacities



Main Outputs:

- baseline_max_flow
- min_cut_analysis, structure.spof_edges/nodes
- sensitivity, failure_impact, parametric_thresholds

Resilience Fit: What IPA Can Do Today

Current, defensible claim:

- Network snapshot analysis for potentially resilience-relevant metrics
- Scenario views in front-end already align with this type of workflow

Clear stress-test outputs for decision support with existing metrics:

- Throughput loss (baseline_max_flow vs scenario)
- Min-cut shifts and SPOF exposure
- Degradation margins and threshold distances
- Multi-failure impact (single-edge, k-edge, etc)

Why this matters for resilience:

- Quantifies where performance collapses
- Identifies fragile structures before failure
- Supports intervention prioritization
- Keeps results auditable and exact per scenario

Important Clarification

- This is not quite a full dynamic resilience model
- Best viewed as a *profiler* on system states

Resilience Extension Path

If system states can be represented as DAG snapshots over time:

$$R = \frac{1}{T} \int_0^T \frac{Q(t)}{Q_0} dt$$

- Use IPA to compute each snapshot performance $Q(t_k)$ exactly
- Aggregate across disturbance/recovery timeline to obtain resilience index
- Exactness remains at snapshot level; index quality depends on time-model assumptions

Practical roadmap:

- 1 Define scenario timeline (shock, degradation, recovery)
- 2 Map each timestep to a DAG state and capacity set
- 3 Run IPA per state, compute $Q(t_k)$ for each state, then aggregate

The Complete IPA Toolkit - From Graph to Insights

Information Propagation Framework

Deployed in Julia Registry | Web Interface Available

